PostgreSQL Lab 4

Q1 Use COALESCE to display each student's nationality. If nationality is NULL, show 'Unknown'.

Q2 Use NULLIF to treat a GPA of 0.0 as NULL. Show student name, their real GPA, and a cleaned version where 0.0 becomes NULL.

Q3 Combine COALESCE + NULLIF: show each student's GPA. If GPA is NULL or 0.0, display 'Not Evaluated'.

Bonus Use NULLIF to calculate average GPA per department, avoiding division by zero. Use COALESCE to replace NULL results with 0. Show dept_name, student count, and safe average GPA.

Q4 Create a temporary table temp_course_stats with: course_code, course_name, enrolled_count, avg_grade. Then find courses where avg_grade is above 75.

Q5 Create a B-tree index on dept_id in the students table.

Q6 Create a UNIQUE index on the email column of students. Then try to insert a duplicate email and observe the error.

Q7 Create a Partial index on salary in professors — only for active professors (is_active = TRUE

Q8 Create a view called v_student_details showing: student_id, full_name, email, gpa, dept_name, faculty_name. Query it to list students in dept_id = 3.

Q9 Create an audit table enrollment_audit. Then create a BEFORE UPDATE trigger on enrollments: if the grade changed, log old_grade, new_grade, student_id, changed_at, changed_by into the audit table.

Q10 Test the grade trigger: update the grade of enrollment_id = 1. Verify the audit log was written. Then update again with the SAME grade and confirm no new audit row.

Q11 Create a BEFORE INSERT trigger on professors: if salary is NULL or below 5000, set it to 5000 automatically.

Q12 Run a transaction that: (1) increases all professor salaries in dept_id=1 by 10%, (2) inserts a log record into a salary_log table. Verify both changes then COMMIT.

```
-- Creating log table first before solving
CREATE TABLE IF NOT EXISTS salary_log (
  log_id    SERIAL PRIMARY KEY,
  prof_id   INTEGER,
  old_salary NUMERIC,
  new_salary NUMERIC,
  changed_by TEXT DEFAULT CURRENT_USER,
  changed_at TIMESTAMPTZ DEFAULT NOW()
);
```

Q13 Demonstrate ROLLBACK: delete all enrollments for student_id=1 inside a transaction, then ROLLBACK. Confirm the rows are still there.

Q14 Use SAVEPOINTs: in one transaction, increase faculty_id=1 budget by 500,000 (save SAVEPOINT), then increase faculty_id=2 budget by 500,000. Undo ONLY the second update using ROLLBACK TO SAVEPOINT, then COMMIT.

Q15 Test SET ROLE: as registrar_user (readwrite), switch to uni_readonly only. Try a SELECT (should work) and an INSERT (should fail). Then RESET ROLE.

Q16 Revoke DELETE on the students table from uni_readwrite. Verify the privilege is gone. Then revoke ALL privileges and remove student_portal from uni_readonly.

Q17 Write the pg_dump commands to: (1) full backup of university_db (3) schema-only backup, (4) data-only backup.

Deadline: four days from now